

Java Polymorphism Upcasting & Downcasting Practical Exercises

Instructions

Complete the following exercises to reinforce your understanding of upcasting and downcasting in Java. Implement your solutions in a Java development environment.

Exercise 1: Demonstrating Upcasting

1. Create a base class `Employee` with:
 - Instance variables `name` and `salary`.
 - A constructor to initialize `name` and `salary`.
 - A method `work()` that prints a generic working message.
2. Create a subclass `Manager` that:
 - Overrides `work()` to include managing responsibilities.
 - Has an additional method `conductMeeting()`.
3. In the `main` method:
 - Create a `Manager` object but store it in an `Employee` variable (**Upcasting**).
 - Call `work()` and observe polymorphic behaviour.

Exercise 2: Demonstrating Downcasting

1. Using the `Manager` object from Exercise 1:
 - Use `instanceof` to check if it's an instance of `Manager`.
 - **Downcast** it back to a `Manager` object.
 - Call `conductMeeting()`.

Exercise 3: Upcasting & Downcasting in Collections

1. Create a **list of Employees** containing both `Employee` and `Manager` objects.
2. Iterate through the list:
 - Call `work()` for each object.
 - If an object is a `Manager`, **downcast** it and call `conductMeeting()`.

Exercise 4: Dynamic Type Checking

1. Modify `Employee`` to be an **abstract class**.
2. Create another subclass `Developer`` that overrides `work()`.
3. In the `main`` method:
 - Store a `Developer`` object in an `Employee`` reference (**Upcasting**).
 - Use `instanceof`` and **Downcasting** to check if it's a `Developer``, then call its methods.

Exercise 5: Advanced Employee Management System

1. Modify `Employee`` to include an **abstract method** `getRole()`.
2. Implement `getRole()` in `Manager`` and `Developer``.
3. In the `main`` method:
 - Create an **array of `Employee`s**.
 - Loop through the array, print each employee's role using **polymorphism**.